EXPERIMENT 1

Design and Configure a Hub-Based Network Using Class A IP Addressing

Objective

The aim of this experiment is to design, configure, and test a basic Local Area Network (LAN) using Cisco Packet Tracer. The network is built using a hub as the central device and five personal computers connected within the same network. Class A IP addressing is used for all systems. This experiment helps in understanding basic networking concepts such as network topology, IP address assignment, and communication between devices in a LAN.

Network Requirements

The network consists of one hub and five personal computers named PC0, PC1, PC2, PC3, and PC4. All the devices are connected to the same hub and belong to a single network.

The Class A IP addressing scheme used is as follows:

- PC0: IP address **56.56.56.1**, Subnet Mask **255.0.0.0**
- PC1: IP address **56.56.56.2**, Subnet Mask **255.0.0.0**
- PC2: IP address **56.56.56.3**, Subnet Mask **255.0.0.0**
- PC3: IP address **56.56.56.4**, Subnet Mask **255.0.0.0**
- PC4: IP address **56.56.56.5**, Subnet Mask **255.0.0.0**

Since all systems are part of the same network and communicate through a hub, a default gateway is not necessary.

Network Topology

The network follows a star topology where all five PCs are connected to a single hub using copper straight-through cables. The hub functions as a central device that forwards incoming data to all connected systems. As a result, all devices share the same broadcast and collision domain, which is a key characteristic of hub-based networks.

Procedure

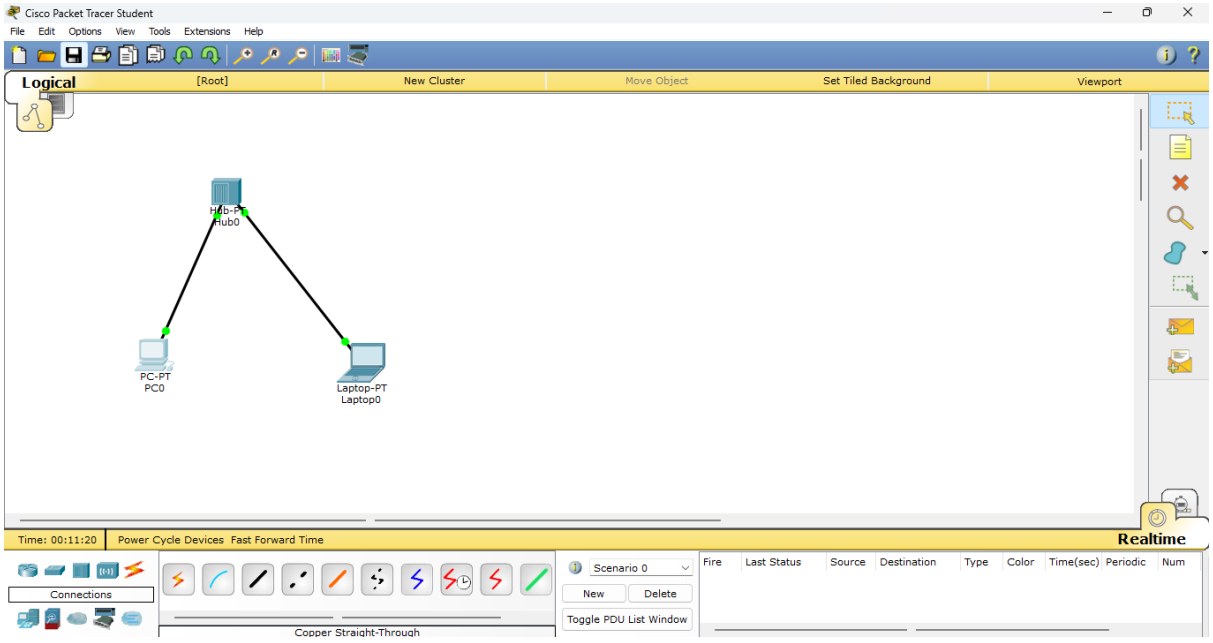
1. Cisco Packet Tracer is launched on the system, and a new project workspace is created.
2. From the **Network Devices** section, a hub is selected and placed at the center of the workspace.
3. Five PCs are then chosen from the **End Devices** section and arranged around the hub for better visibility.
4. Using the **Connections** tool, copper straight-through cables are selected. Each PC's FastEthernet0 port is connected to an available port on the hub.

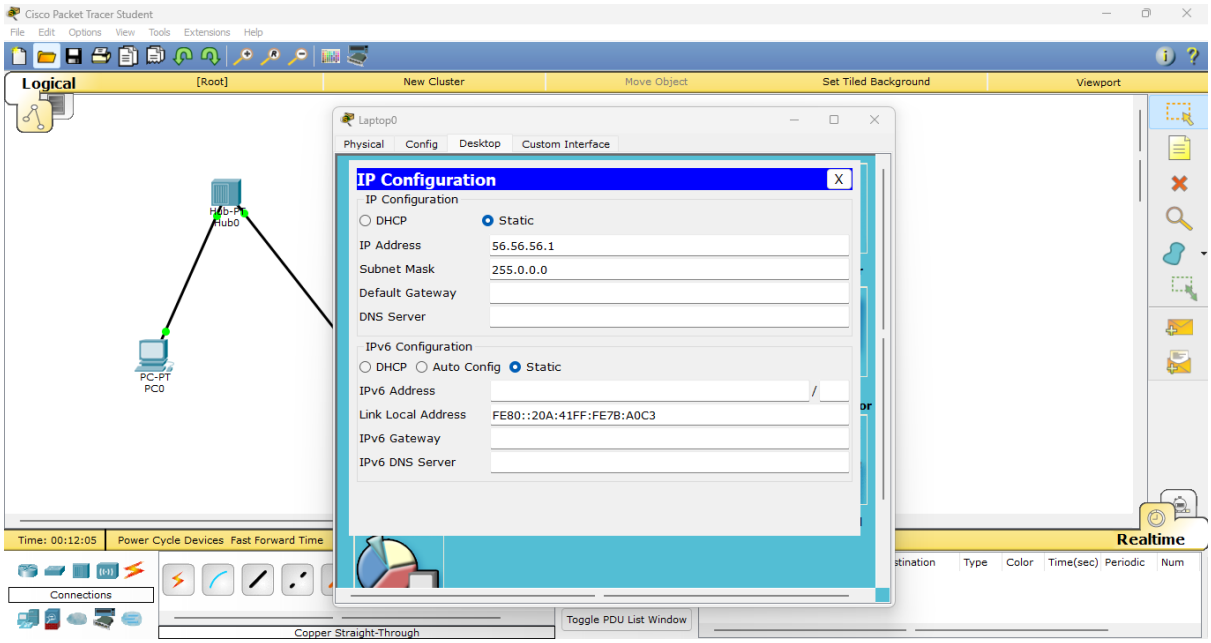
5. After making the connections, the link lights are checked to confirm that all devices are properly connected.
6. IP configuration is then performed on each PC.
    - PC0 is selected, the **Desktop** tab is opened, and **IP Configuration** is chosen.
    - The IP address **56.56.56.1** and subnet mask **255.0.0.0** are entered.
    - The same steps are repeated for PC1 to PC4 using their respective IP addresses.
7. No default gateway is entered since the network does not include a router.
8. Once all IP addresses are assigned, network connectivity is tested.
9. The **Command Prompt** is opened on one PC, and the ping command is used to check communication with the other PCs.
10. This process is repeated from different PCs to ensure full connectivity across the network.
11. After successful verification, the Packet Tracer file is saved for submission.

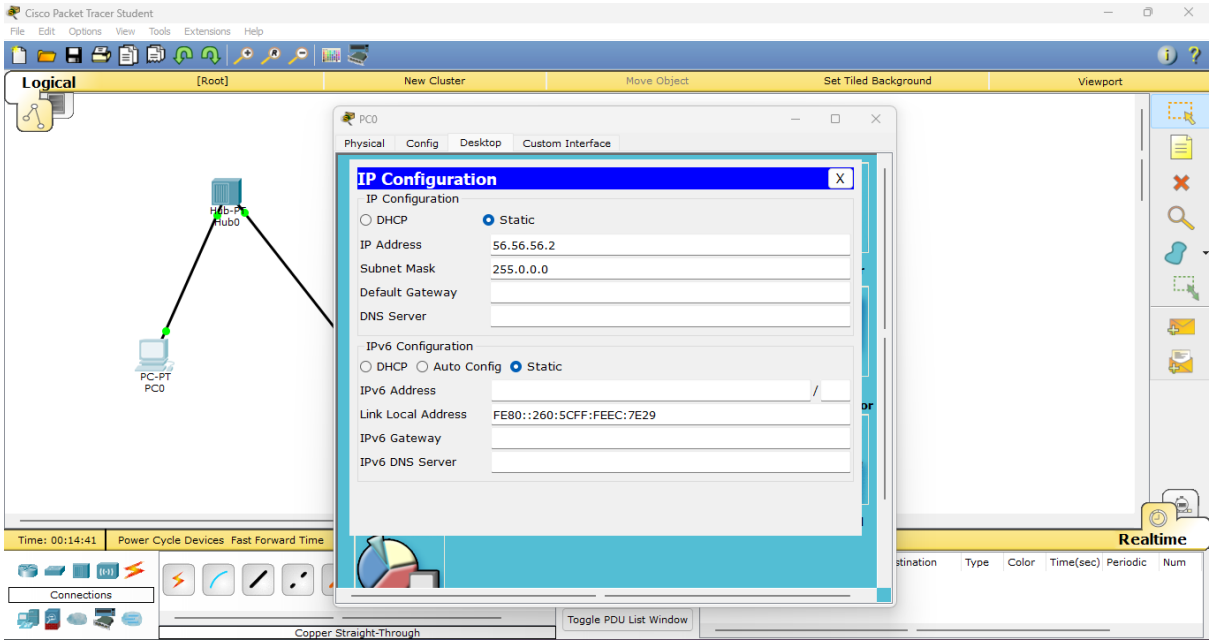
Verification

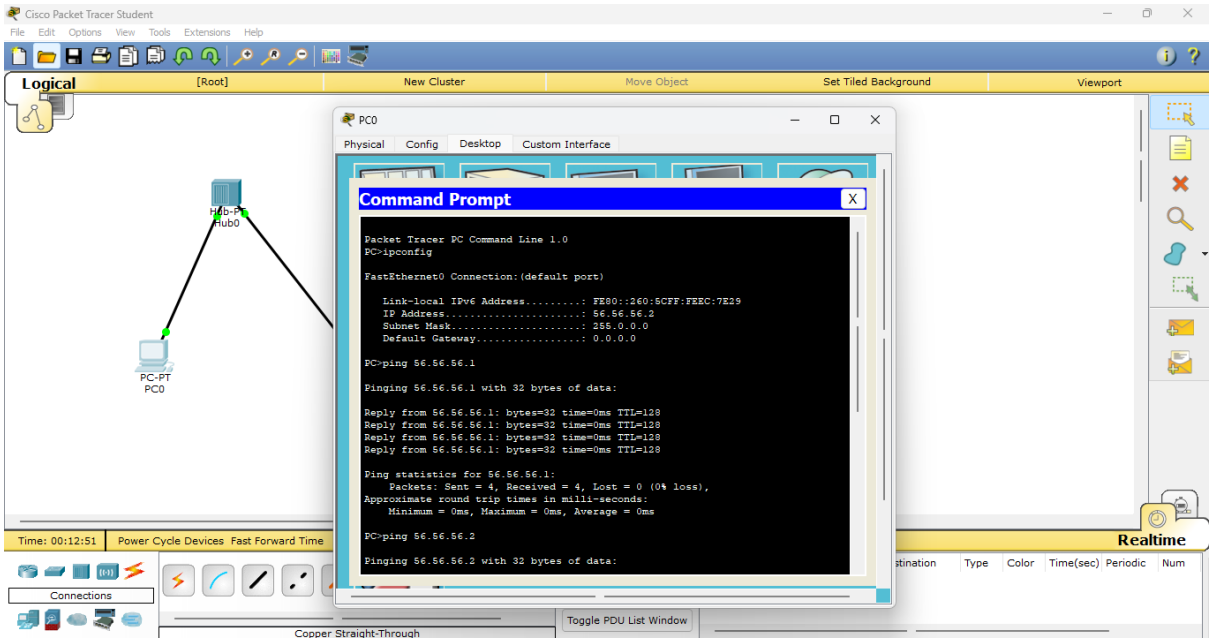
Network verification is carried out using ICMP echo requests. The ping command is executed between all PCs. Receiving successful replies from each system confirms that the IP addressing, cabling, and hub connections are correctly configured.

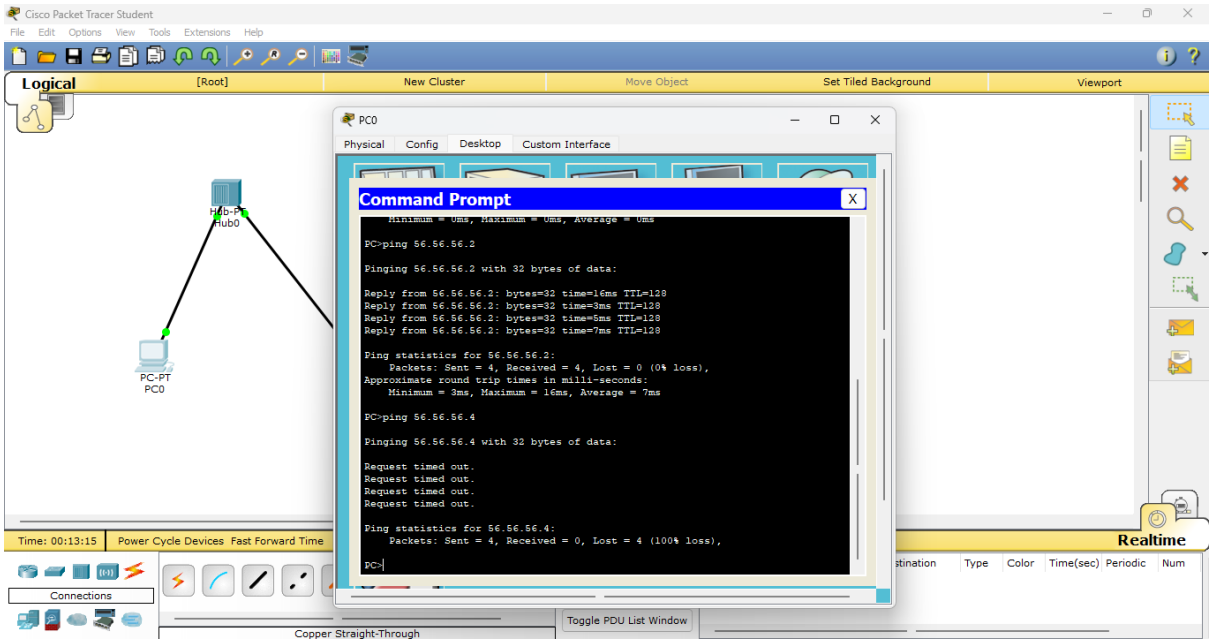
OUTPUT:




Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
Laptop0
Physical
Config
Desktop
Custom Interface
IP Configuration
IP Configuration
DHCP
Static
IP Address
56.56.56.1
Subnet Mask
255.0.0.0
Default Gateway
DNS Server
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::20A:41FF:FE7B:A0C3
IPv6 Gateway
IPv6 DNS Server
Hub0
PC-PT
PC0
Time: 00:12:05
Power Cycle Devices
Fast Forward Time
Realtime
Connections
Copper Straight-Through
Toggle PDU List Window


Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
PC0
Physical
Config
Desktop
Custom Interface
IP Configuration
IP Configuration
DHCP
Static
IP Address
56.56.56.2
Subnet Mask
255.0.0.0
Default Gateway
DNS Server
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::260:5CFF:FEEC:7E29
IPv6 Gateway
IPv6 DNS Server
Hub0
PC-PT
PC0
Time: 00:14:41
Power Cycle Devices
Fast Forward Time
Realtime
Connections
Copper Straight-Through
Toggle PDU List Window





Successful ping responses are observed between all PCs, indicating proper communication within the network.

Summary

In this experiment, a hub-based LAN was successfully designed and implemented using Cisco Packet Tracer. Five PCs were connected to a single hub and configured with Class A IP addresses in the 56.56.56.0 network. All devices were able to communicate without errors, demonstrating correct IP configuration and effective data transmission through a hub. This experiment provides a clear understanding of basic LAN setup, Class A addressing, and hub-based network communication.

EXPERIMENT 2

Design and Configure a Switch Network Using Cisco Packet Tracer

---

Aim

To design and configure a switch-based network using Cisco Packet Tracer by connecting five PCs to a switch, assigning Class A IP addresses, and verifying network connectivity using the ping command.

Objective

To understand the working of a switch and to establish communication between multiple computers in a LAN using proper IP addressing.

Summary

In this experiment, a switch-based network was created using Cisco Packet Tracer. Five PCs were connected to a switch, assigned IP addresses in the same subnet, and connectivity was verified using the ping command.

Apparatus / Software Required

- **Cisco Packet Tracer**
- **1 Switch-PT**
- **5 PCs (PC0–PC4)**
- **Copper Straight-Through cables**
- **PT-SWITCH-NM-1CFE module (if additional ports are required)**

Network Topology

(A star topology is used where all PCs are connected to a central switch.)

[Insert Screenshot 1: Physical topology showing switch connected to five PCs]

IP Addressing Scheme

Device	IP Address	Subnet Mask
PC0	10.1.1.1	255.0.0.0
PC1	10.1.1.2	255.0.0.0
PC2	10.1.1.3	255.0.0.0
PC3	10.1.1.4	255.0.0.0
PC4	10.1.1.5	255.0.0.0

(Default gateway is not required since no router is used.)

Procedure

1. **Open Cisco Packet Tracer.**
2. **Place one Switch-PT in the workspace.**
3. **If the switch does not have enough ports:**
    - **Click the switch and open the Physical tab.**
    - **Power OFF the switch.**
    - **Insert PT-SWITCH-NM-1CFE module.**
    - **Power ON the switch.**
4. **Place five PCs in the workspace.**
5. **Connect each PC to the switch using Copper Straight-Through cables.**
6. **Configure IP addresses:**
    - **Open each PC → Desktop → IP Configuration.**
    - **Enter the IP address and Subnet Mask as per the table.**
7. **Verify connectivity:**
    - **Open Command Prompt in any PC.**
    - **Use the ping command to test connectivity with other PCs.**

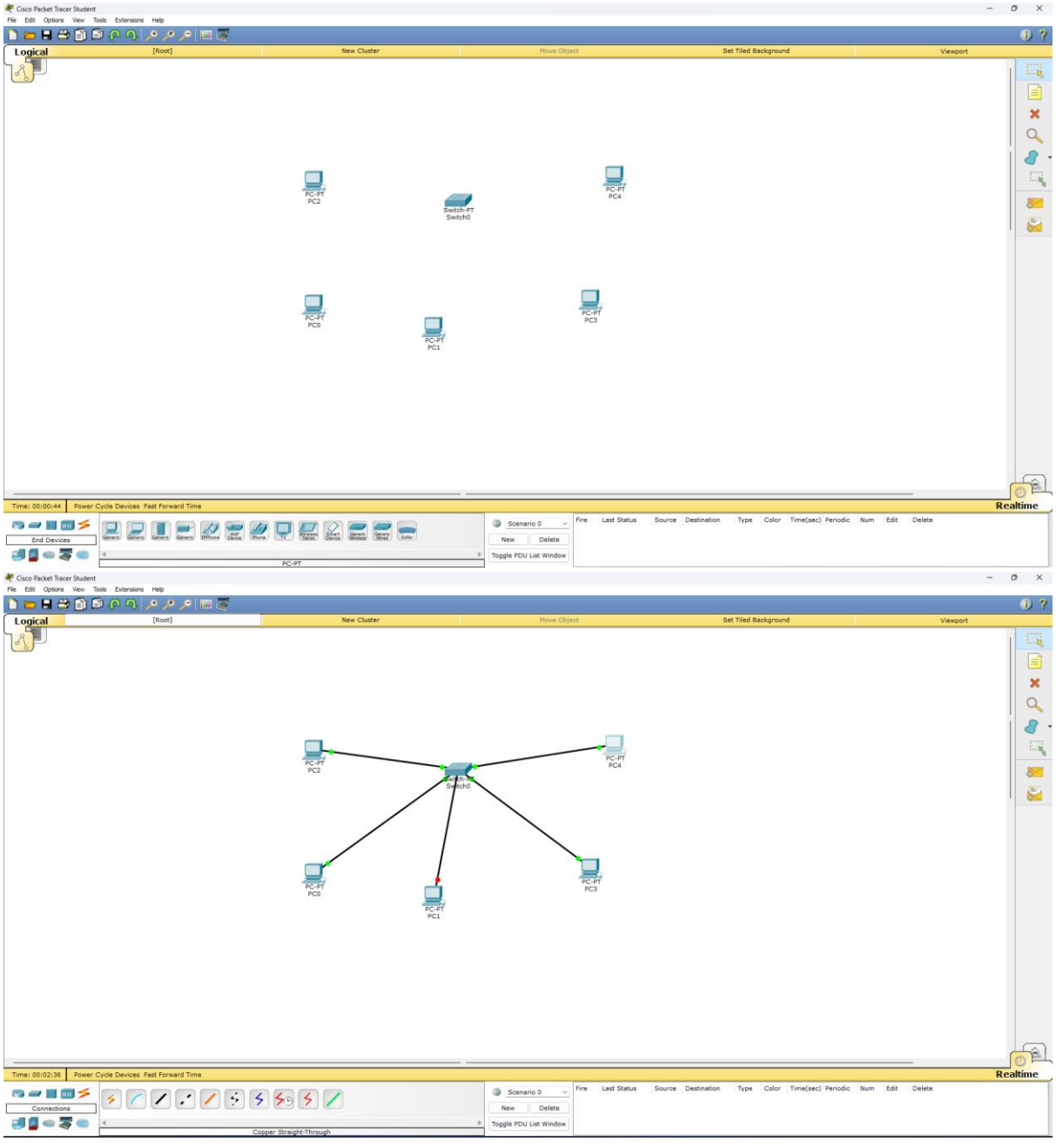
Result

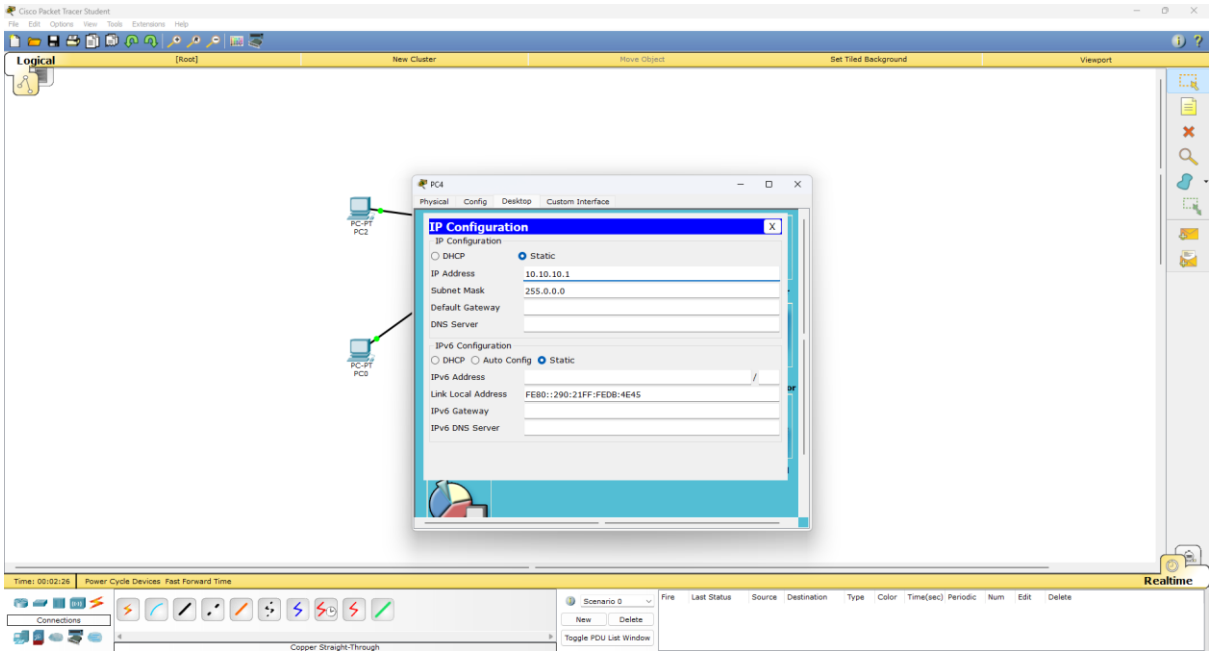
All PCs successfully communicated with each other through the switch. The network was configured correctly and connectivity was verified using the ping command.

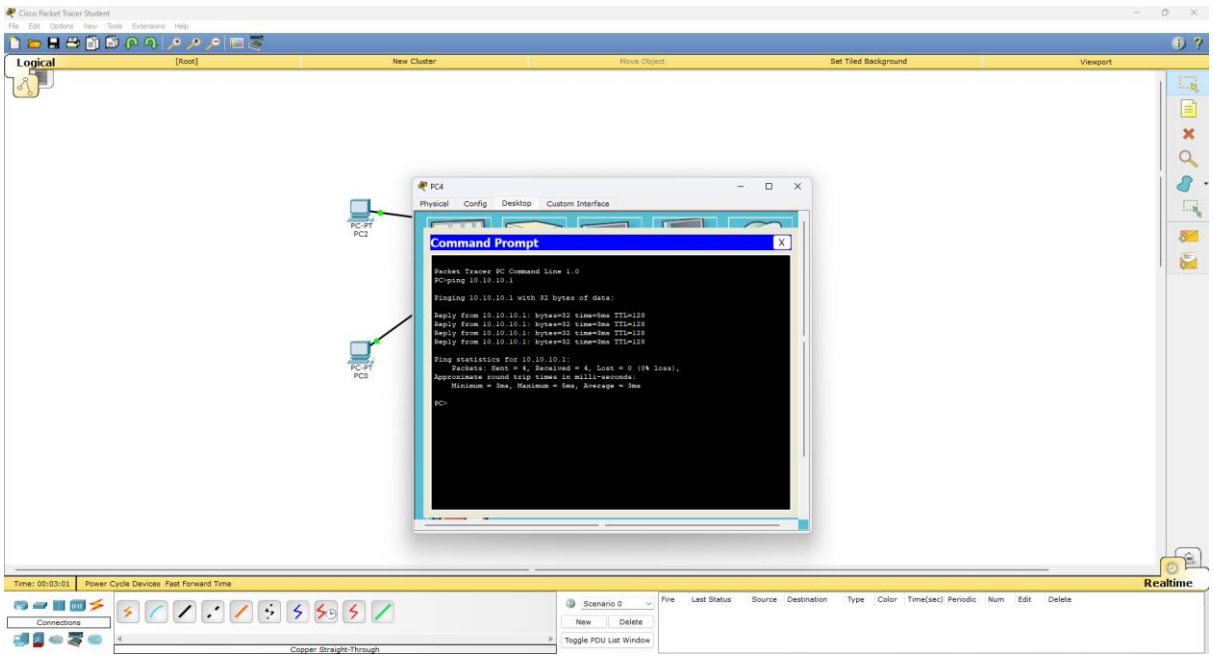
Conclusion

The experiment verified the operation of a switch in a LAN environment. The switch forwards frames intelligently to the destination port, improving network performance compared to a hub.

OUTPUT:




Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
PC4
Physical
Config
Desktop
Custom Interface
IP Configuration
IP Configuration
DHCP
Static
IP Address
10.10.10.1
Subnet Mask
255.0.0.0
Default Gateway
DNS Server
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::290:21FF:FEDB:4E45
IPv6 Gateway
IPv6 DNS Server
PC-PT PC2
PC-PT PC0
Time: 00:02:26
Power Cycle Devices
Fast Forward Time
Realtime
Connections
Copper Straight-Through
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete


Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
PC4
Physical
Config
Desktop
Custom Interface
Command Prompt
Packet Tracer PC Command Line 1.0
PC>ping 10.10.10.1
Pinging 10.10.10.1 with 32 bytes of data:
Reply from 10.10.10.1: bytes=32 time=5ms TTL=128
Reply from 10.10.10.1: bytes=32 time=3ms TTL=128
Reply from 10.10.10.1: bytes=32 time=2ms TTL=128
Reply from 10.10.10.1: bytes=32 time=3ms TTL=128
Ping statistics for 10.10.10.1:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 2ms, Maximum = 5ms, Average = 3ms
PC>
PC-PT PC2
PC-PT PC0
Time: 00:03:01
Power Cycle Devices
Fast Forward Time
Realtime
Connections
Copper Straight-Through
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete

EXPERIMENT 3

Design a Virtual LAN for LAB 1.a and LAB 2.b

Aim

To design and configure a Virtual LAN (VLAN) in Cisco Packet Tracer for the AIML and CSD faculty blocks so that both departments are logically in separate LANs with different broadcast domains while maintaining inter-PC communication within their VLANs.

Objective

- **To understand the concept and configuration of VLANs.**
- **To separate broadcast domains for different departments (AIML and CSD).**
- **To verify connectivity among PCs within the same VLAN using PDU packets.**

Summary

In this experiment, VLANs were created for two departments (AIML and CSD) in separate blocks (Block-A and Block-B). Switches were configured to assign ports to VLANs, logically grouping faculty PCs while keeping broadcast traffic separated. Connectivity within each VLAN was tested using PDU packets in Cisco Packet Tracer, confirming that VLAN configuration was successful.

Apparatus / Software Required

- **Cisco Packet Tracer**
- **2 Switches (for Block-A and Block-B)**
- **6–8 PCs (3–4 PCs per department)**
- **Copper Straight-Through cables**

Network Topology

- **Block-A: Faculty PCs of AIML connected to Switch-A**
- **Block-B: Faculty PCs of CSD connected to Switch-B**
- **Switch-A and Switch-B interconnected to maintain communication through VLANs if required**

[Insert Screenshot 1: Physical topology showing PCs connected to two switches]

VLAN Design and IP Addressing

Department	VLAN ID	Switch Ports Assigned	Sample IP Address	Subnet Mask
AIML	10	Switch-A: Fa0/1–Fa0/3	192.168.10.1–3	255.255.255.0
CSD	20	Switch-B: Fa0/1–Fa0/3	192.168.20.1–3	255.255.255.0

VLAN 10 is for AIML faculty
VLAN 20 is for CSD faculty
Each VLAN forms a separate broadcast domain

Procedure

1. **Open Cisco Packet Tracer.**
2. **Place two switches representing Block-A and Block-B.**
3. **Place PCs for AIML in Block-A and PCs for CSD in Block-B.**
4. **Connect each PC to its respective switch using Copper Straight-Through cables.**
5. **Configure VLANs on the switches:**
    - **Select a switch → CLI → Enter VLAN configuration mode**
    - **Create VLAN 10 for AIML:**

```
Switch> enable
Switch# configure terminal
Switch(config)# vlan 10
Switch(config-vlan)# name AIML
Switch(config-vlan)# exit
```

    - **Assign ports to VLAN 10:**

```
Switch(config)# interface range fa0/1 - 3
Switch(config-if-range)# switchport mode access
Switch(config-if-range)# switchport access vlan 10
```

    - **Repeat for VLAN 20 for CSD on Switch-B.**
6. **Assign IP addresses to each PC according to the table.**
7. **Interconnect Switch-A and Switch-B using a trunk port (if communication between VLANs is needed).**

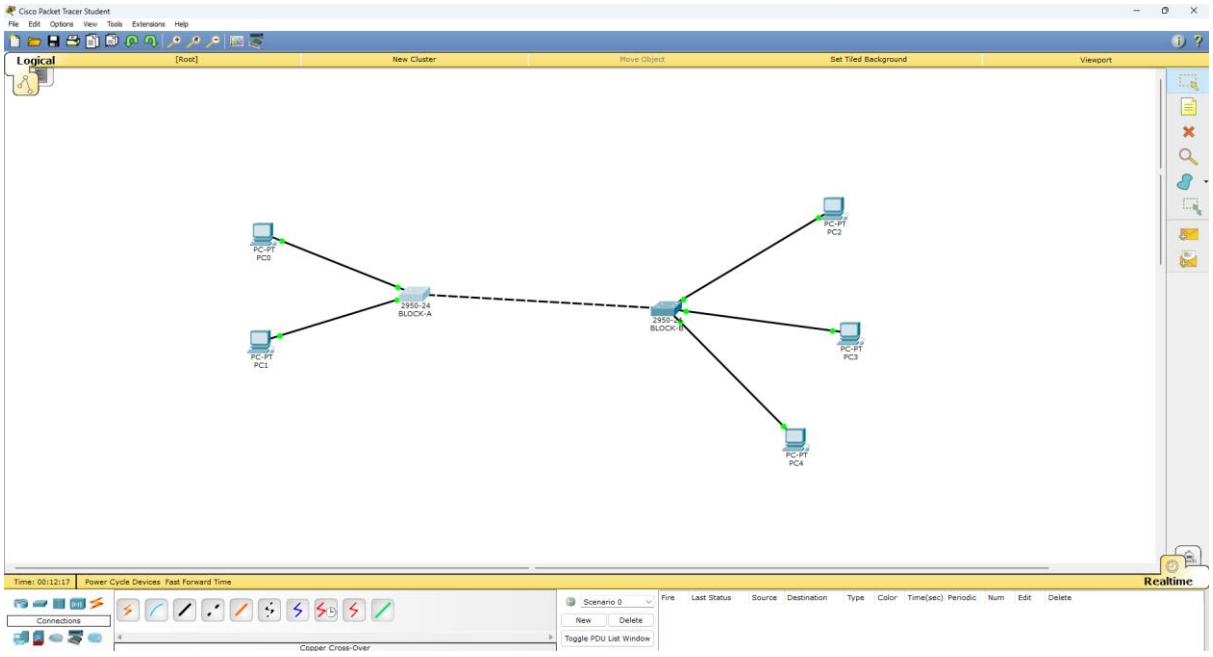
8. **Test connectivity:**
    - **Use PDU in Packet Tracer → Select a PC → Send PDU to other PCs in the same VLAN**
    - **Ensure packets are received successfully.**

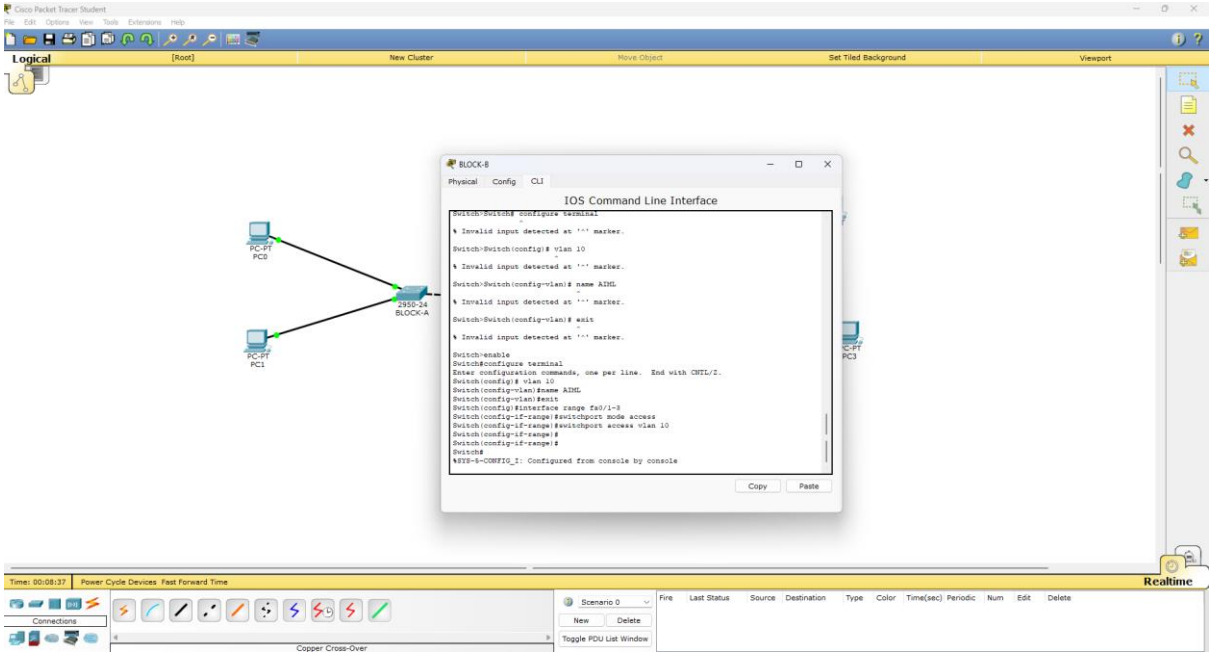
Result

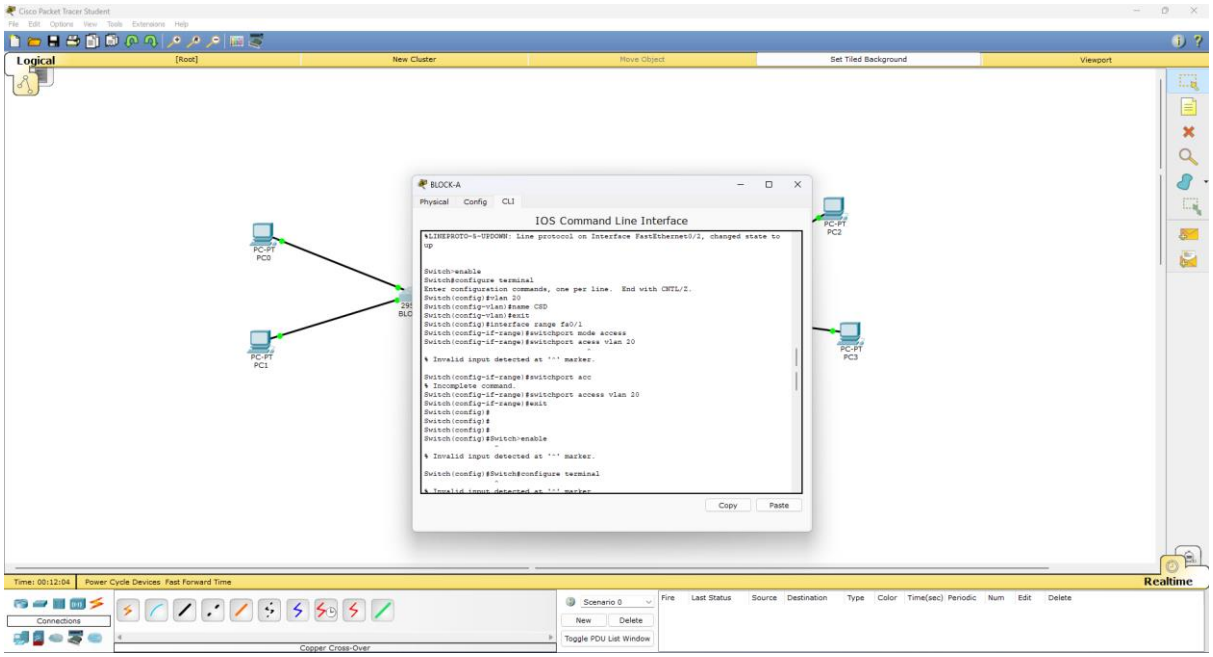
- **VLANs were successfully created for AIML and CSD faculty.**
- **PCs within the same VLAN communicated successfully.**
- **Broadcast domains were separated; PDU packets did not cross VLAN boundaries unless routed through a Layer 3 device (optional).**

Conclusion

The experiment demonstrated the creation and configuration of VLANs to logically separate different departments in a campus network. VLANs allowed separation of broadcast domains while maintaining intra-department connectivity. Testing with PDU packets confirmed that VLAN configuration was correct and efficient for network segmentation.




Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
BLOCK-B
Physical
Config
CLI
IOS Command Line Interface
Switch>Switch$ configure terminal
% Invalid input detected at '^' marker.
Switch>Switch(config)# vlan 10
% Invalid input detected at '^' marker.
Switch>Switch(config-vlan)# name AIML
% Invalid input detected at '^' marker.
Switch>Switch(config-vlan)# exit
% Invalid input detected at '^' marker.
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)# vlan 10
Switch(config-vlan)#name AIML
Switch(config-vlan)#exit
Switch(config)#interface range fa0/1-3
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport access vlan 10
Switch(config-if-range)#
Switch(config-if-range)#
Switch#
%SYS-5-CONFIG_I: Configured from console by console
Copy
Paste
PC-PT PC0
PC-PT PC1
2950-24 BLOCK-A
PC-PT PC3
Time: 00:08:37
Power Cycle Devices
Fast Forward Time
Realtime
Connections
Copper Cross-Over
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete


Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
BLOCK-A
Physical
Config
CLI
IOS Command Line Interface
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/2, changed state to up
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 20
Switch(config-vlan)#name CSD
Switch(config-vlan)#exit
Switch(config)#interface range fa0/1
Switch(config-if-range)#switchport mode access
Switch(config-if-range)#switchport acess vlan 20
% Invalid input detected at '^' marker.
Switch(config-if-range)#switchport acc
% Incomplete command.
Switch(config-if-range)#switchport access vlan 20
Switch(config-if-range)#exit
Switch(config)#
Switch(config)#
Switch(config)#
Switch(config)#Switch>enable
% Invalid input detected at '^' marker.
Switch(config)#Switch#configure terminal
% Invalid input detected at '^' marker.
Copy
Paste
PC-PT PC3
PC-PT PC1
PC-PT PC2
PC-PT PC3
Time: 00:12:04
Power Cycle Devices
Fast Forward Time
Realtime
Connections
Copper Cross-Over
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete

EXPERIMENT - 4

EXPERIMENT - 4

Aim

To capture, identify, and analyze various network protocols using Wireshark and to study at least five important protocols (TCP, UDP, ARP, ICMP, HTTP) in detail using display filters and flow graphs.

Requirements

- System with Internet connectivity
- Wireshark installed
- Web browser
- Command Prompt / Terminal

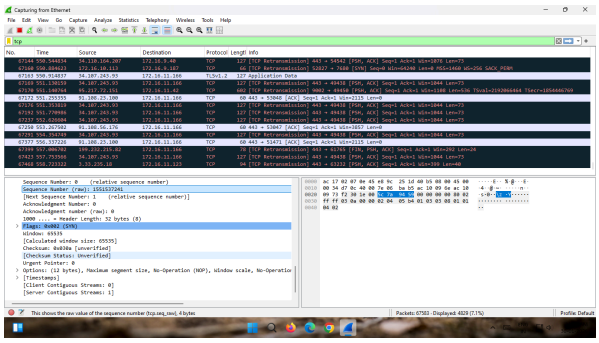
Procedure

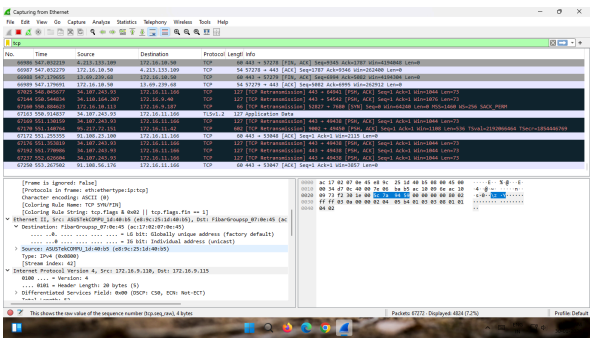
1. Install and open Wireshark on the system.
2. Select the active network interface (Wi-Fi/Ethernet).
3. Click on **Start Capture** to begin packet capturing.
4. Generate network traffic by browsing a website or using commands like `ping` and `nslookup`.
5. Stop the packet capture after sufficient packets are recorded.
6. Apply display filters such as `tcp`, `udp`, `arp`, `icmp`, and `http` to analyze specific protocols.
7. Select individual packets to examine header details in the packet details pane.
8. Use **Follow** → **TCP Stream** to observe complete TCP communication.

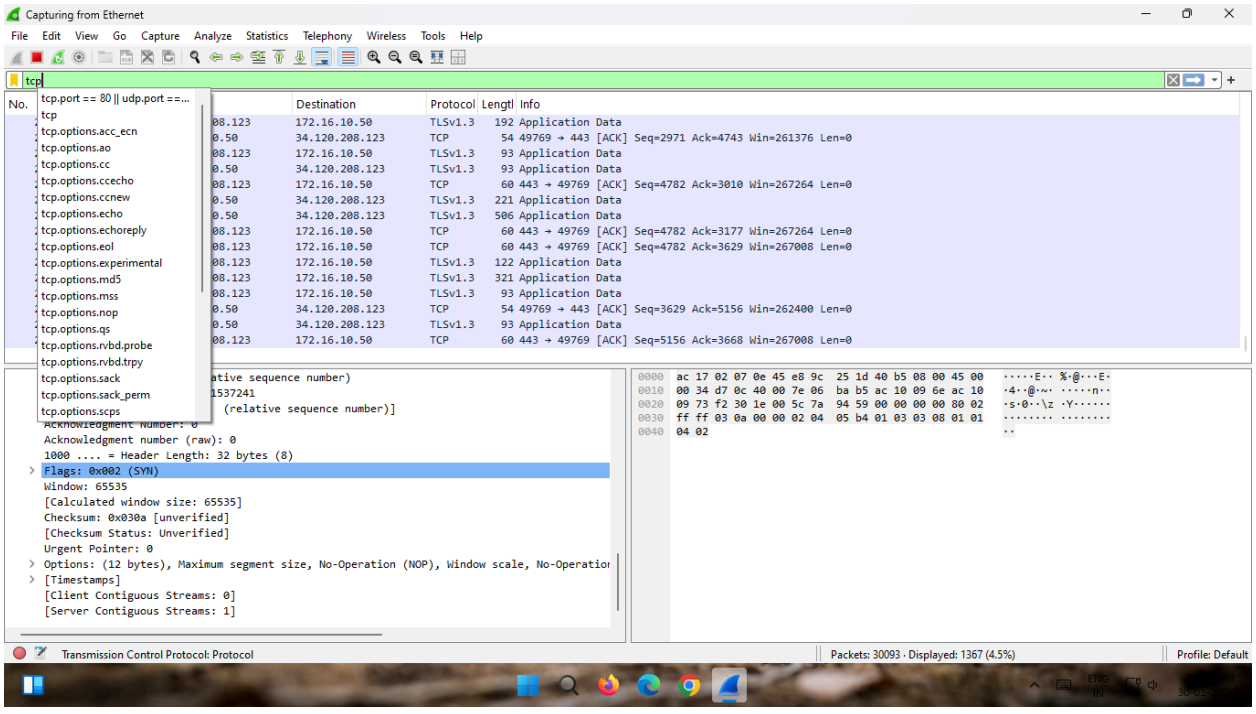
9. Open **Statistics → Flow Graph** to visualize packet flow between source and destination.

10. Analyze the captured packets and record observations for each protocol.

OUTPUT







Observation

- TCP showed three steps before sending data (SYN, SYN-ACK, ACK).
- UDP sent data directly without any connection setup.
- ARP asked for a MAC address and received a reply.
- ICMP showed request and reply packets during ping.
- HTTP showed website request and server response.

RESULT

The network protocols TCP, UDP, ARP, ICMP, and HTTP were successfully captured and analyzed using Wireshark. The packet details, filters, and flow graph helped in understanding how data is transmitted between source and destination. The working and behavior of each protocol were studied clearly and verified.

EXPERIMENT - 5

EXPERIMENT - 5

Aim

To design a Hamming Code sender utility and to generate an error-detecting bitstream using even parity.

Procedure

1. Input the binary data string.
2. Calculate required check bits using 2r≥m+r+12^r \geq m + r + 12r≥m+r+1.
3. Place check bits at positions 1, 2, 4, 8, etc.
4. Insert data bits in remaining positions.
5. Assign each data bit to check bit groups based on binary position.
6. Compute check bits to maintain even parity.
7. Combine all bits to form final encoded bitstream.

Program

```
def calculate_parity_bits(m):

    p = 0

    while (2 ** p) < m + p + 1:

        p += 1

    return p



def position_parity_bits(data, p):

    j = 0

    k = 1

    m = len(data)
```

```
    result = ''

    for i in range(1, m + p + 1):
        if i == 2 ** j:
            result += '0'
            j += 1
        else:
            result += data[-k]
            k += 1
    return result[::-1]

def calculate_parity_values(code, p):
    n = len(code)
    for i in range(p):
        val = 0
        for j in range(1, n + 1):
            if j & (2 ** i) == (2 ** i):
                val = val ^ int(code[-j])
        code = code[:n-(2**i)] + str(val) + code[n-(2**i)+1:]
    return code

def detect_correct_error(code, p):
    n = len(code)
```

```
    error_pos = 0
    for i in range(p):
        val = 0
        for j in range(1, n + 1):
            if j & (2 ** i) == (2 ** i):
                val = val ^ int(code[-j])
        error_pos += val * (2 ** i)

    if error_pos:
        print(f"Error found at position: {error_pos}")
        corrected_code = code[:n - error_pos] + str(1 - int(code[n -
error_pos])) + code[n - error_pos + 1:]
        return corrected_code

    else:
        print("No error detected.")
        return code

def main():
    original_data = input("Enter the data bits: ")
    m = len(original_data)
    p = calculate_parity_bits(m)
    positioned_data = position_parity_bits(original_data, p)
    hamming_code = calculate_parity_values(positioned_data, p)
```

```
    print(f"Hamming Code: {hamming_code}")

    received_code = input("Enter the received Hamming Code: ")

    corrected_code = detect_correct_error(received_code, p)

    print(f"Corrected Hamming Code: {corrected_code}")



if __name__ == "__main__":

    main()
```

Output

```
Enter the data bits: 1001010

Hamming Code: 10011010001

Enter the received Hamming Code: 10011010001

No error detected.

Corrected Hamming Code: 10011010001
```

Observation

1. The data length increases from 4 bits to 7 bits due to added check bits.
2. Check bits are placed at positions 1, 2, and 4, forming a structured pattern.
3. Each check bit covers multiple data bits, creating overlapping groups.
4. Even parity maintains a balanced number of 1s in each group.
5. The final encoded output is `0110011`, ensuring error detection capability.

Result

Hamming code successfully generated with even parity.

Experiment-6

Experiment-6

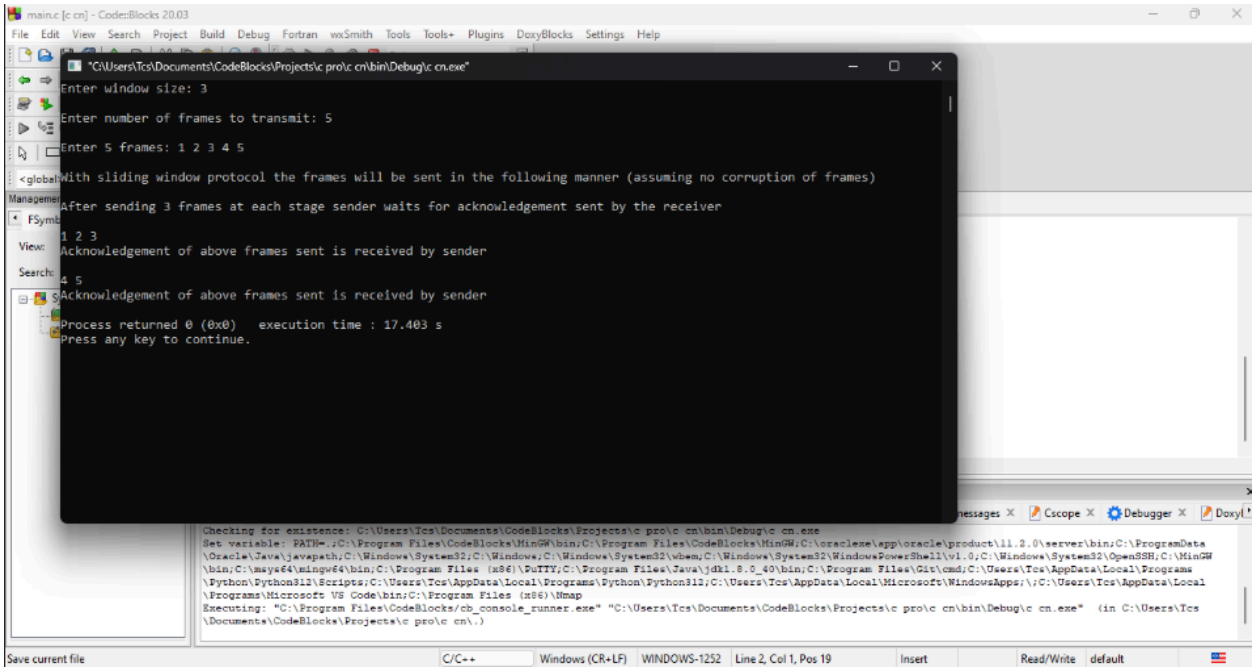
Aim

To simulate the **Sliding Window Protocol** for data transmission.

Procedure

- Start the program.
- Input the **window size (w)**.
- Enter the **number of frames (f)** to be transmitted.
- Input the sequence of frames.
- The sender transmits frames in groups equal to the window size.
- After sending each window, the sender waits for acknowledgement.
- Continue until all frames are transmitted.

Output



Observation

- Frames are sent in groups based on window size.
- Acknowledgement is received after each group of frames.
- Sender waits only after sending a full window.
- Remaining frames are sent if frames are not multiple of window size.
- Transmission efficiency is improved compared to one-by-one sending.

Result

Sliding Window Protocol is successfully simulated with efficient frame transmission using grouped acknowledgements.

Experiment-7

Experiment-7

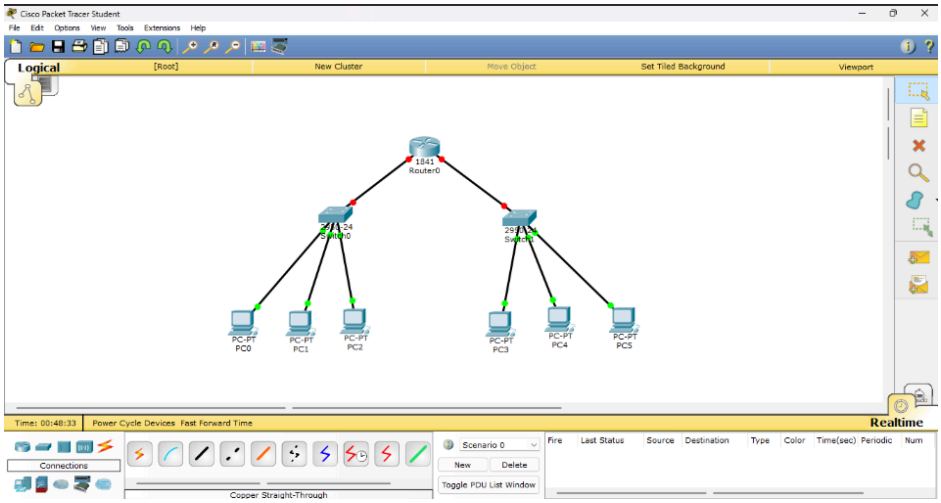
Aim

To design and configure **2 subnets** from the given network 192.168.10.0/25.

Procedure

1. Given network: **192.168.10.0/25** (128 IP addresses).
2. Divide the network into **2 equal subnets**.
3. Subnet mask becomes **/26 (255.255.255.192)**.
4. Subnet 1:
    - Network ID: **192.168.10.0/26**
    - Usable IPs: 192.168.10.1 – 192.168.10.62
5. Subnet 2:
    - Network ID: **192.168.10.64/26**
    - Usable IPs: 192.168.10.65 – 192.168.10.126
6. Assign at least 3 IP addresses to computers in each subnet.
7. Configure IP address and subnet mask in each system.
8. Use a router or switch to connect subnets.
9. Send PDU (ping) packets to test connectivity.

Output



Observation

1. The given network is successfully divided into two equal subnets.
2. Each subnet supports more than 3 computers.
3. Devices within the same subnet communicate directly.
4. Communication between subnets requires routing.
5. PDU packets confirm successful data transmission.

Result

Thus Subnets are successfully created and configured.

Experiment-8

Experiment-8

Aim

To implement an Echo Client-Server communication using TCP and UDP sockets in Java, where the client sends a message and the server echoes it back.

Procedure

- Start the program execution.
- Create a server program:
  - For TCP, initialize a `ServerSocket` with a port number.
  - For UDP, initialize a `DatagramSocket` with a port number.
- Wait for client communication:
  - In TCP, accept the client connection using `accept()`.
  - In UDP, wait to receive a packet using `receive()`.
- Create a client program:
  - For TCP, establish connection using `Socket` with server IP and port.
  - For UDP, create a `DatagramSocket` and prepare a packet with server address.
- Send message from client to server.
- Receive the message at the server side.
- Echo the same message back to the client:
  - In TCP, send via output stream.
  - In UDP, send using `DatagramPacket`.
- Receive echoed message at the client side and display it.
- Repeat communication if required until termination.
- Close all sockets and end the program.

T3client.java

File Edit View

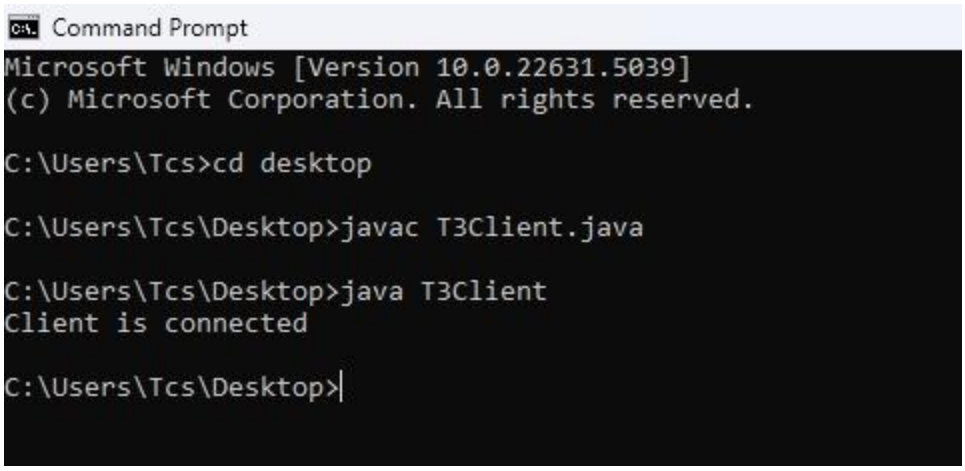
```
import java.net.*;
import java.io.*;
import java.util.*;
public class T3Client
{
    public static void main(String[] args) {
        try{
            Socket Client =new Socket("localhost",5000);
            System.out.println("Client is connected");
        }catch (Exception e){
            System.out.println(e);
        }
    }
}
```

T3server.java

File Edit View

```
import java.net.*;
import java.io.*;
import java.util.*;
class T3Server {
    public static void main(String[] args) {
        try
        {
            ServerSocket ss =new ServerSocket(5000);
            System.out.println("waiting for Client");
            Socket server = ss.accept();
            System.out.println("Server is connected");
        } catch (Exception e) {
                System.out.println(e);
        }
    }

}
```



Observation

- In **TCP**, connection is established before communication.
- Messages are delivered reliably and in order.
- In **UDP**, no connection setup is required.
- Data may be faster but not guaranteed to reach.
- Both programs successfully echoed client messages.

Result

The Echo Client-Server programs using both **TCP and UDP sockets** were successfully implemented and executed. The server correctly received messages from the client and sent back the same message as output.

Experiment-9

Experiment-9

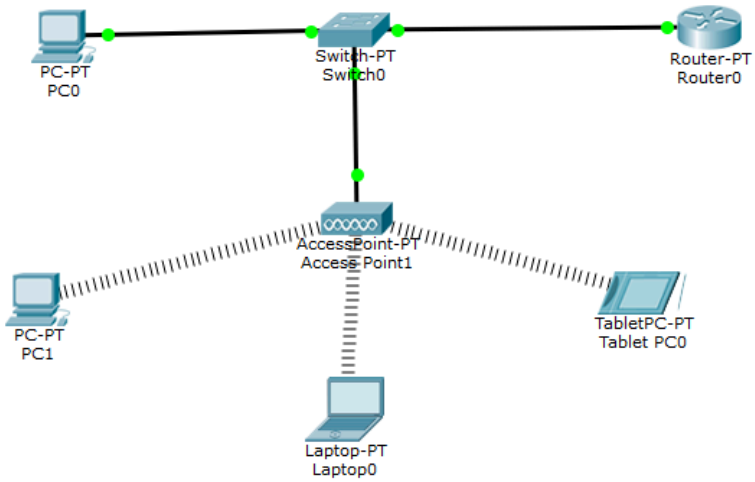
Aim

To study and configure a wireless router and establish a wireless network connection between devices.

Procedure

- Switch on the wireless router and connect it to the power supply.
- Connect the router to the internet source (modem or ISP cable).
- Use a computer/mobile device to search for available Wi-Fi networks.
- Connect to the router's default SSID (network name).
- Open a web browser and enter the router's IP address (e.g., 192.168.1.1).
- Log in using default username and password.
- Configure basic settings:
    - Change SSID (network name).
    - Set a secure password (WPA2/WPA3).
- Save settings and restart the router if required.
- Reconnect devices using the new SSID and password.
- Test the connection by accessing websites or transferring data.

Output



Observation

- The wireless router broadcasts an SSID that devices can detect.
- Devices successfully connected using the configured password.
- Data transmission occurred without physical cables.
- Network speed and signal strength varied based on distance and obstacles.

Result

The wireless router was successfully configured, and a stable wireless network connection was established between devices, enabling internet access and communication.